UNISINOS

Nossa sala de aula é o mundo.

TRABALHO 3, APRESENTAÇÃO INTERMEDIÁRIA, TIME 04

App de gerenciamento de convites

Convite por link, confirmação de presença e restrições alimentares. Evolução da implementação desde o Grau A.



EQUIPE

Andreas Grings, Gabriel Tomasi
Guilherme Toebe, Tiago Passos

DISCIPLINA

Arquitetura de Software
Prof. Kleinner Farias

DATA

27/10/2026

O problema e os usuários

- Quem organiza uma festa costuma juntar as confirmações pelo WhatsApp e perde a conta de quantas pessoas vêm, quantos acompanhantes cada uma leva e o que cada uma não pode comer.
- **Anfitriã:** cria o convite, compartilha o link e acompanha as respostas.
- **Convidado:** abre o link e responde sem criar conta.
- **Buffet:** recebe a lista de restrições alimentares para planejar o cardápio.

Escopo do MVP

1 HISTÓRIA

Conta do anfitrião

Criar conta e entrar.

3 HISTÓRIAS

Criação do convite

Criar o convite, escolher template e cores e ver a prévia.

2 HISTÓRIAS

Compartilhamento

Gerar o link, publicar e despublicar.

3 HISTÓRIAS

Confirmação

Responder sem conta, informar acompanhantes e alterar pelo link pessoal.

1 HISTÓRIA

Restrições alimentares

Informar a restrição. Alergia exige descrição.

2 HISTÓRIAS

Painel

Lista de presença e consolidação das restrições, com exportação em CSV.

As doze histórias estão implementadas, integradas e rodando pelo docker compose.

Linha do tempo do projeto

Sprint 1
18/08 a 31/08
Requisitos e casos de uso

Sprint 2
01/09 a 14/09
Arquitetura e decisões

Sprint 3
15/09 a 28/09
Documento de arquitetura e protótipo

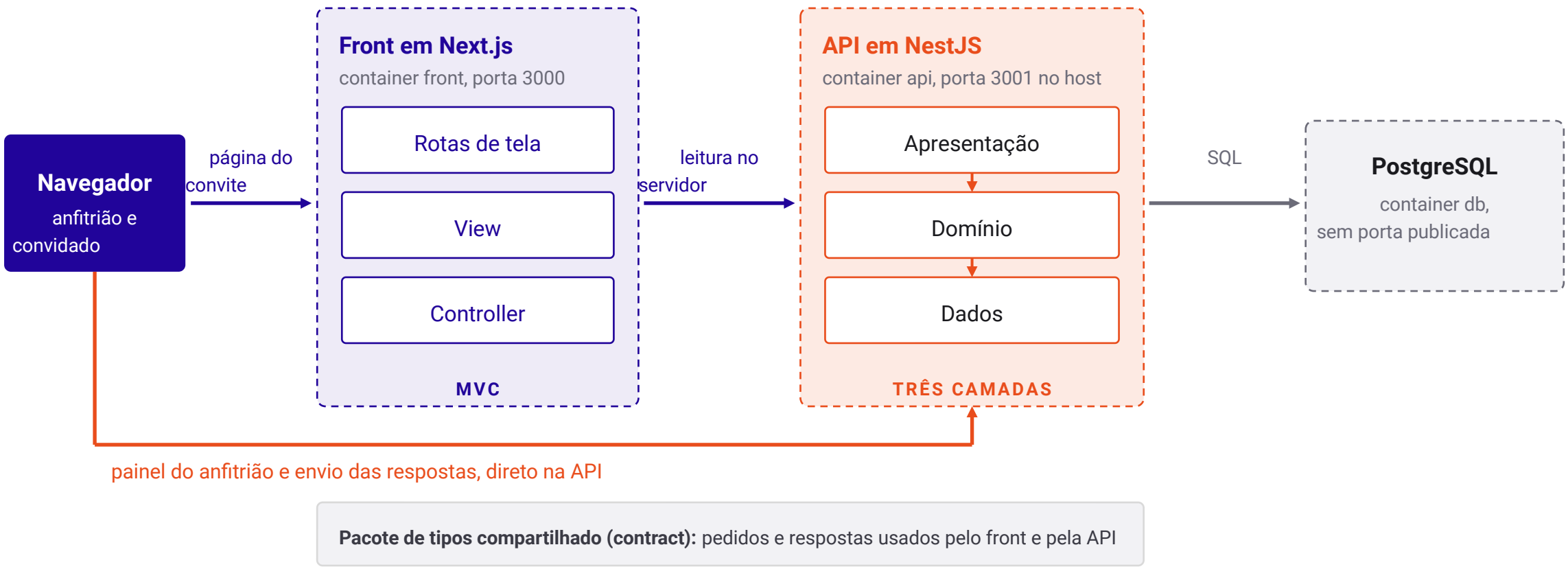
Sprint 4
06/10 a 19/10
Base do projeto

Sprint 5
20/10 a 02/11
Integração do front e do back

Sprint 6
03/11 a 16/11
Consolidação e Pitch Final

Sprints 1 a 3, Grau A: documentação e protótipo. Sprints 4 a 6, Grau B: implementação.

Tiers e camadas



Pacote de tipos compartilhado (contract): pedidos e respostas usados pelo front e pela API

Base do projeto: lint, integração contínua e Docker

LINT E INTEGRAÇÃO CONTÍNUA

- O lint recusa import que quebra a regra de camadas: o Domínio não importa da Apresentação, e os Dados não importam de nenhuma camada de cima.
- Em todo PR, a integração contínua roda lint, formatação, tipos, testes contra um PostgreSQL e build.
- A mesma integração constrói as imagens da API, do front e do mock.

DOCKER COMPOSE

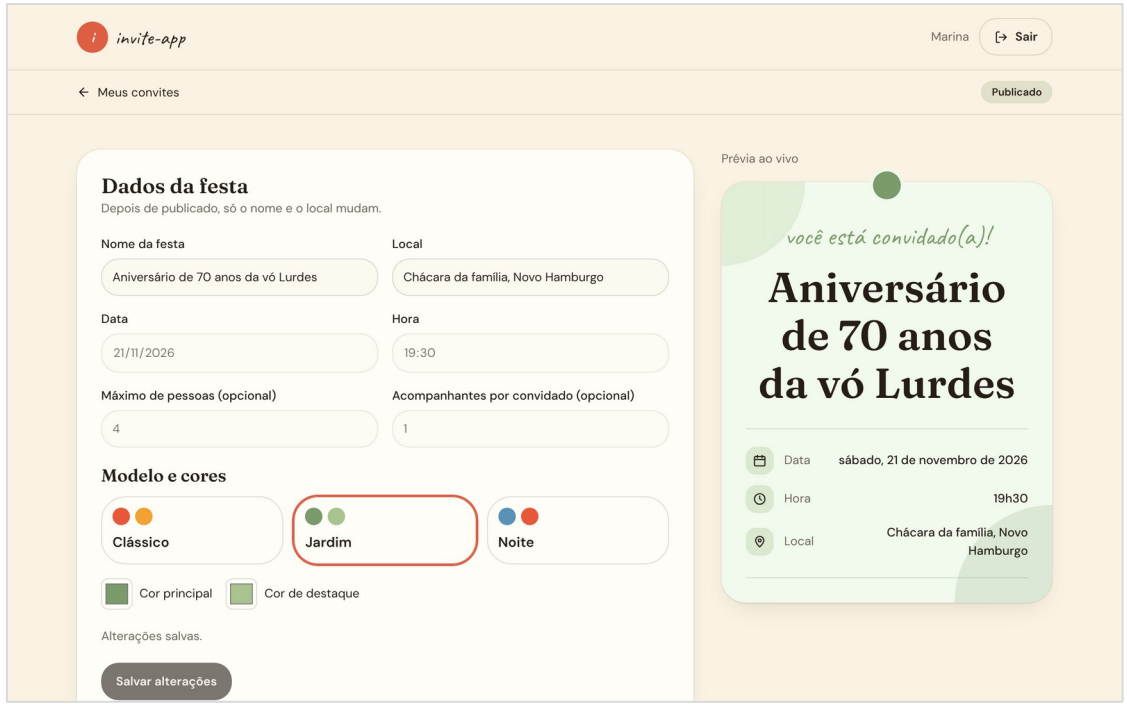
- Três serviços: banco, API e front. O banco não publica porta.
- O front publica a 3000 e a API a 3001, porque o navegador chama a API direto no painel e no envio da resposta.
- A API só sobe depois que o banco responde ao healthcheck.
- Um segundo arquivo troca a API por um mock, para trabalhar nas telas sem banco.

Regras da arquitetura no código

REGRA	ONDE FICA
Quem não tem acesso não descobre se um convite existe	No Domínio e na Apresentação. Link inválido, convite despublicado e convite de outro anfitrião dão o mesmo 404, com o mesmo corpo e cabeçalho, conferido por um teste de ponta a ponta.
Sessão do anfitrião em cookie assinado	Na Apresentação. O front não guarda token.
Link do convite gerado no Domínio	No Domínio, com gerador seguro, na primeira publicação. Nunca no navegador.

A regra do teto de vagas está no bloco do back-end.

Telas do front-end



Editor do convite, com a prévia



Convite no celular



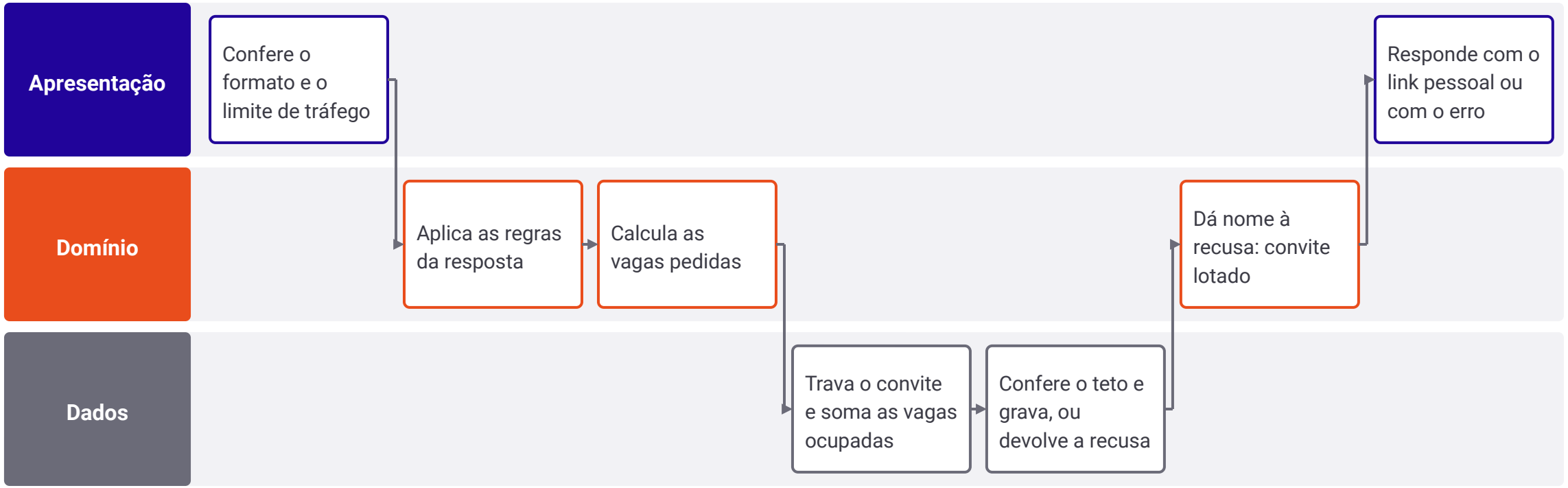
Resposta enviada, com o link pessoal

> As telas partiram do protótipo e foram adequadas à arquitetura: o visual foi mantido, reescrito em CSS puro, e as regras de negócio saíram da tela.

Mudanças no front para seguir a arquitetura

PONTO	NO PROTÓTIPO	HOJE
Vagas na página pública	Mostrava quantas vagas restavam	Sem contagem. A lotação aparece só quando a resposta é recusada
CSV para o buffet	Montado no navegador	Gerado pela API, com proteção contra fórmula de planilha
Sessão do anfitrião	Token guardado no navegador	Cookie de sessão. O front não guarda token
Limites do formulário	Conferidos só na tela	A tela limita os campos, e a API valida

Caminho de uma resposta na API



> “Sim” ocupa uma vaga para o convidado e uma para cada acompanhante. “Talvez” e “Não” não ocupam vaga. “Não” zera acompanhantes e observação. Alergia exige descrição.

Testes

DOMÍNIO

Regras

Vagas pedidas por resposta, o não vou zerando acompanhantes e observação, descrição da alergia, comparação por dia no fuso do projeto e publicação.

BANCO

Contra PostgreSQL

Gravação dentro do teto e concorrência: quatro respostas simultâneas pedindo duas vagas cada, num teto de cinco. Passam duas.

ROTAS

Ponta a ponta

O fluxo completo pelas rotas, e o mesmo 404 para link inválido, convite despublicado e convite de outro anfitrião.

Os testes rodam na integração contínua em todo PR, com um PostgreSQL de serviço.

Roteiro da demonstração

PASSO	O QUE MOSTRA	QUEM FALA
Criar, personalizar e publicar	Template, cores, prévia e o link gerado	Guilherme
Responder como convidado	Resposta sem conta, com acompanhante e restrição alimentar	Gabriel
Responder num convite lotado	A recusa pelo teto de vagas	Gabriel
Alterar pelo link pessoal	A troca da resposta para talvez	Gabriel
Painel e CSV	Lista de presença e restrições exportadas	Guilherme
Despublicar	O link do convite deixa de abrir	Guilherme

Organização do trabalho

- Sprints de duas semanas, acompanhadas no quadro Kanban do Azure Boards. Cada história é quebrada em tarefas de tela e de API, com responsável.
- Conversas pelo WhatsApp e no tempo de aula. As decisões ficam registradas na documentação do repositório.
- Integração por PR no GitHub. Um PR só entra na main com a integração contínua aprovada.

Dificuldades encontradas

INTEGRAÇÃO

Trabalho feito em paralelo

O front e o back foram escritos fora da main e precisaram ser adequados ao pacote de tipos e às camadas para entrar.

FRONT-END

Regras no protótipo

O protótipo nasceu antes de a arquitetura estar no código. Contagem de vagas, CSV e token ficavam na tela.

BACK-END

Regra na camada errada

A regra das vagas estava na consulta da camada de Dados. O cálculo passou para o Domínio.

Lições: definir os tipos compartilhados antes das telas, integrar front e back antes do fim da sprint, e manter as regras de negócio no Domínio desde a primeira versão.

Planejamento até a apresentação final, em 17/11

- Revisar os artefatos das Sprints 1 a 3 contra o código, ligando cada história ao caso de uso, à rota, à tela e ao teste.
- Desenhar o diagrama de sequência da lista de presença, que faltou no Grau A.
- Completar o protótipo de baixa fidelidade da Sprint 3 a partir das telas reais.
- Dar o aval do time às decisões novas da arquitetura.
- Preparar e ensaiar o Pitch Final.

TIME 04

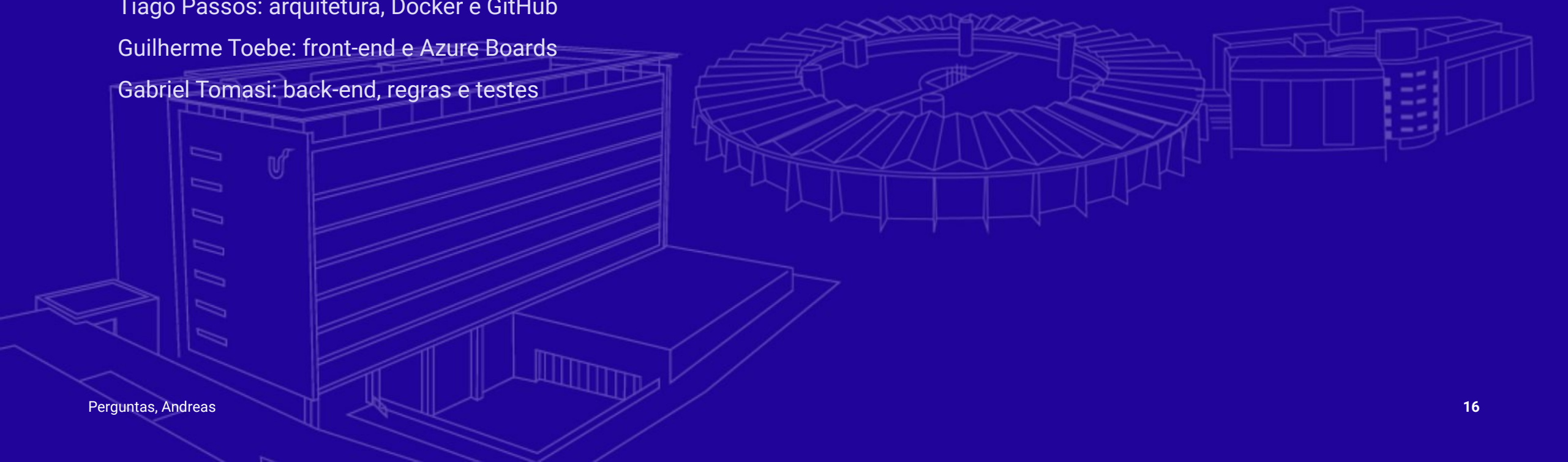
Perguntas

Andreas Grings: problema, requisitos e planejamento

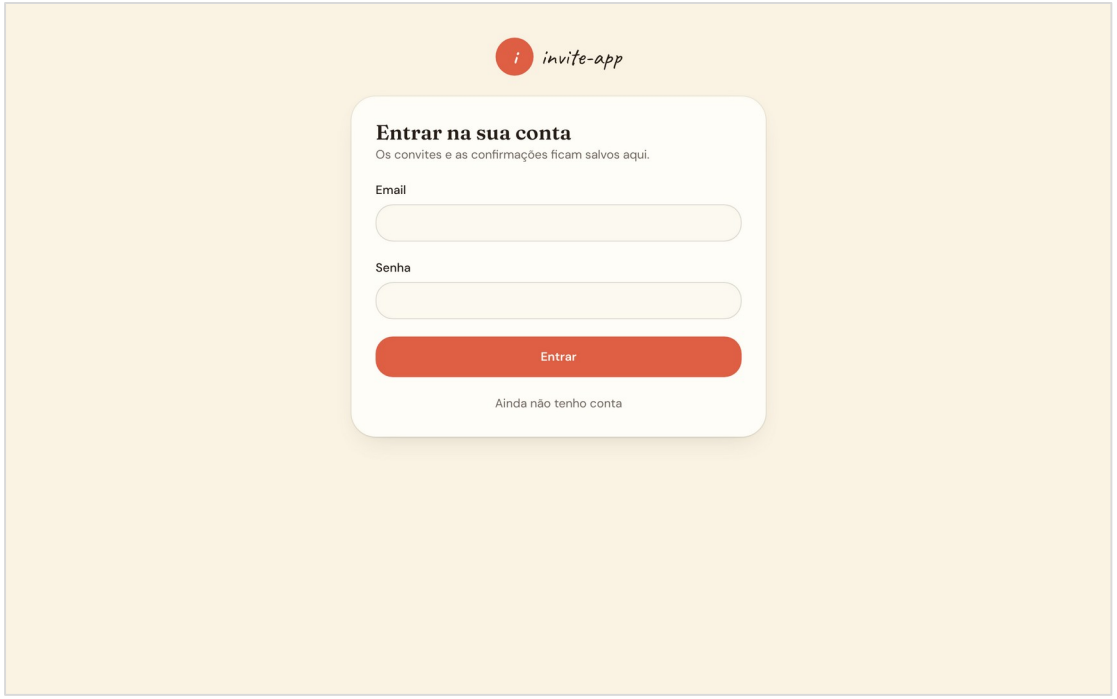
Tiago Passos: arquitetura, Docker e GitHub

Guilherme Toebe: front-end e Azure Boards

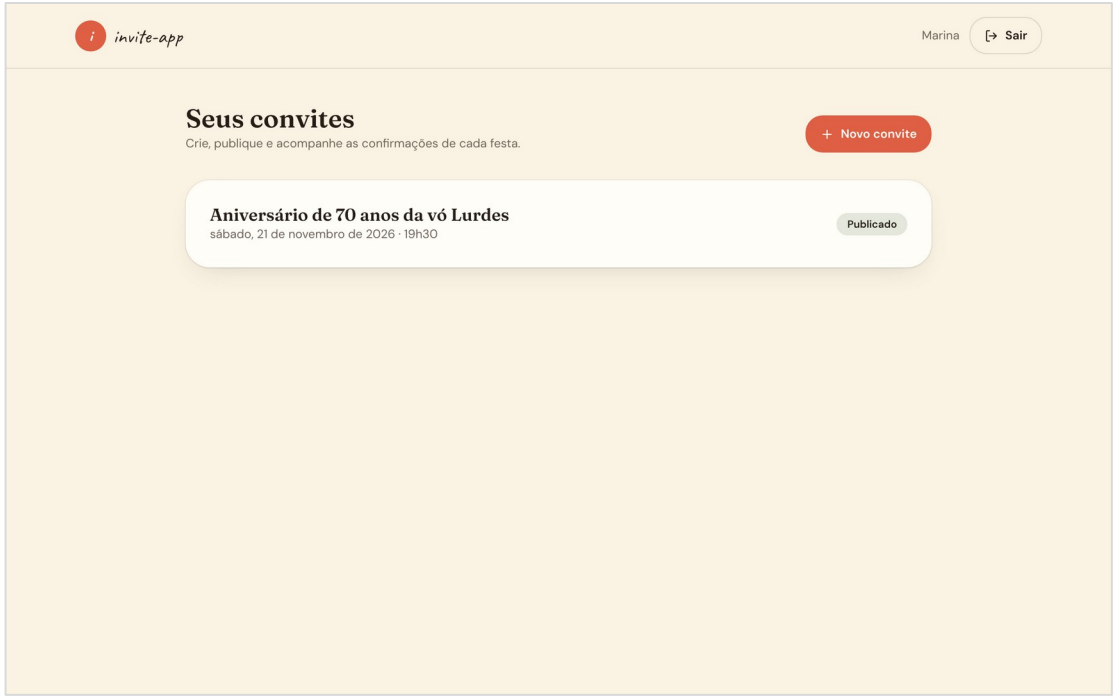
Gabriel Tomasi: back-end, regras e testes



Telas do anfitrião



Tela de entrada, com o acesso ao cadastro



Lista de convites

Capturas da aplicação rodando localmente em 26/10/2026.

Telas do convidado



Convite



Resposta enviada



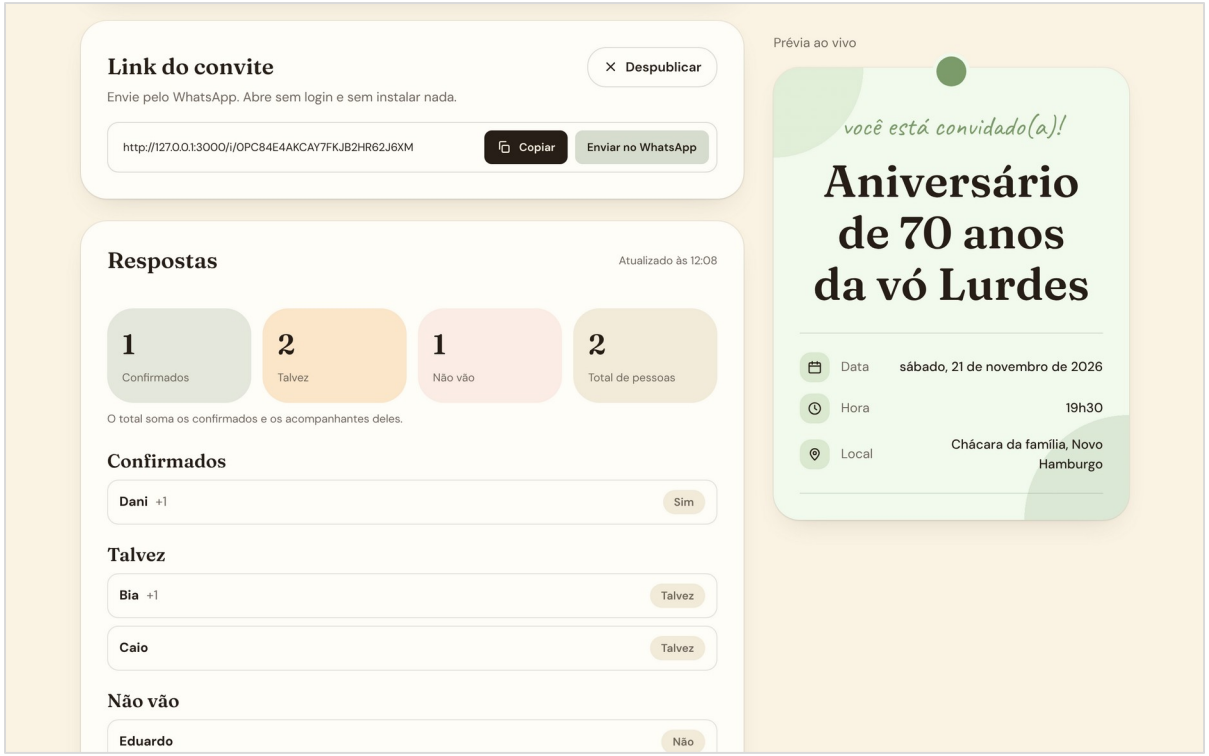
Convite lotado



Link pessoal

Capturas da aplicação rodando localmente em 26/10/2026.

Painel e convite despublicado



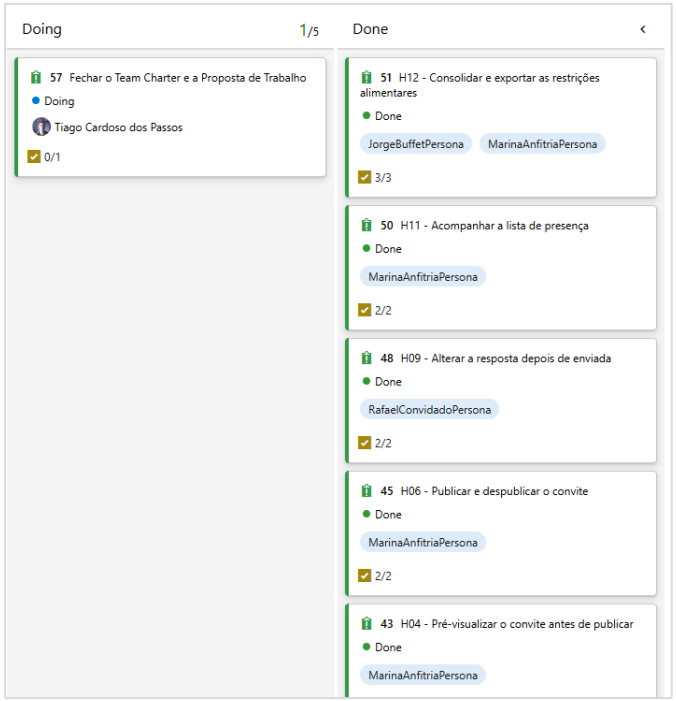
Painel de respostas



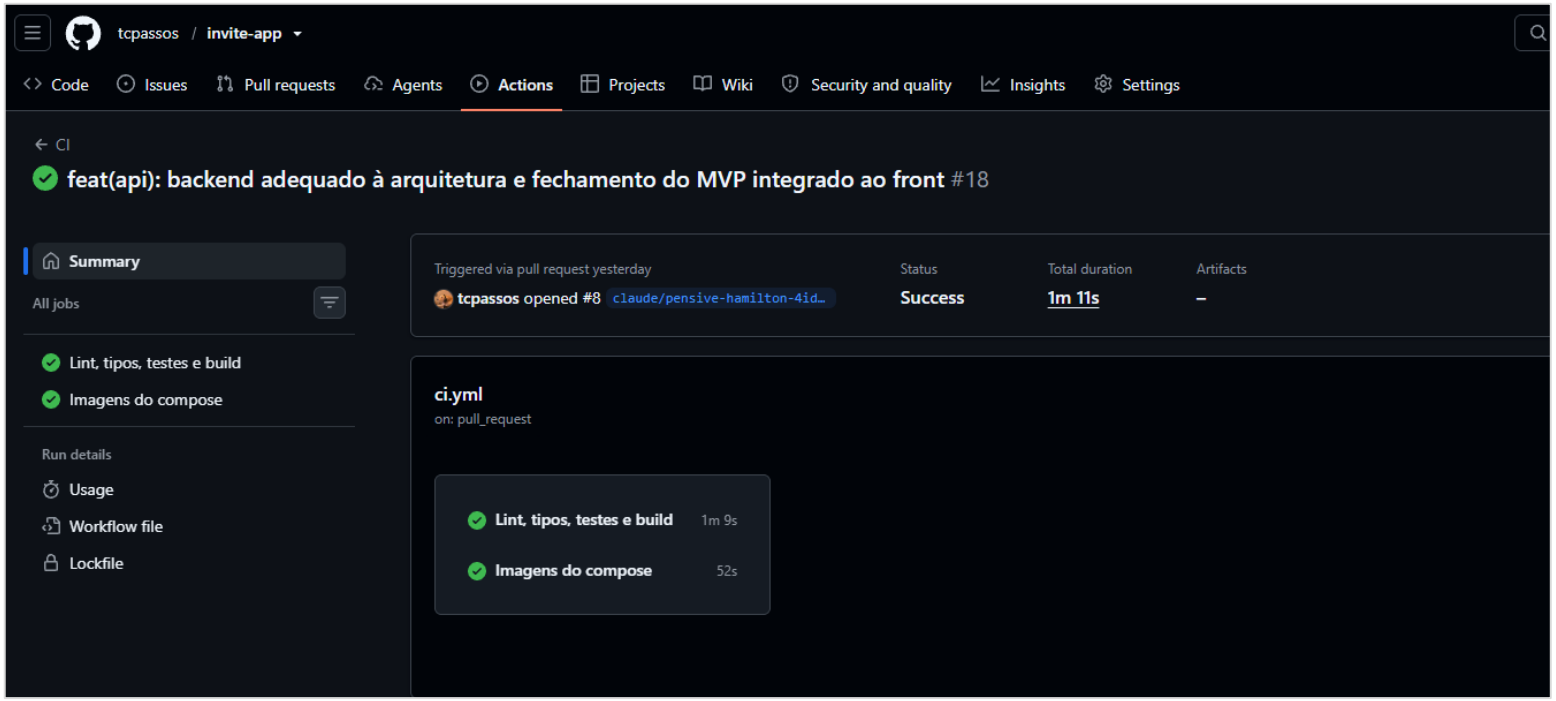
Link do convite depois de despublicar

Capturas da aplicação rodando localmente em 26/10/2026.

Quadro Kanban e integração no GitHub



Quadro Kanban no Azure Boards



Integração contínua aprovada no PR do back-end

Capturas do Azure Boards e do GitHub em 26/10/2026.